Introduction

Cool. Hello, everyone, and welcome to lecture 3 of CME 295. So today is a very exciting day, because we're going to finally introduce large language models. But I guess before we go into that I'm just going to start, traditionally, with some announcements. So some of you wanted to have the slides before the class. So just a heads up that in case you want to use the slides to do some annotations, they're on the website right now. So feel free to get them. And with Shervin, we'll try to, just on a regular basis, every Thursday evening, have them publish on the website so that you can download them and annotate. Cool. So with that, let's start. And as usual, we're going to recap last week's episodes. So if you remember, lecture 1 and lecture 2 were all about introducing the concept of self-attention

Recap of Transformers-based models

and linking them to the construct of the transformer. And what we did last lecture was look at all the types of models that are out there and how they were all based on the transformer. So there are three categories, three main categories of models. So the first one that we saw was encoder-decoder model, which basically relies on the transformer. It has the encoder of the transformer, the decoder of the transformer. And typically, the tasks there are input text. So text in, text out. So we saw that one example was T5 and all the variations. The second type of model is where we remove the decoder from the transformer and we obtain an encoder-only model. So we went deeper into BERT, which is the typical encoder-only model. And I guess, we also saw that what BERT has is this nice property that it's encoded embeddings are very meaningful and expressive of the inputs. And so we saw the example of classification, sentiment extraction. So what we did was, in particular, take into consideration the encoded embedding of the CLS token. So I guess in real life, BERT is used to encode documents to encode sentences. And we're going to see later in the class how these models are useful. And then last but not least, we have the third category of models, which is decoder only. So we only keep the decoder part of the transformer. And so here, we also do a little bit of a modification. So we remove the cross-attention because we don't need it anymore. We don't have an encoder. And these kinds of models are text in, text out. And so GPT is a very good example of such models. And actually, most models these days, they're decoder-only. So these are the three main kinds of models that you can see out there. So far so good for everyone?

LLM definition

Cool. So with that, I'm going to introduce the term LLM. So LLM stands for large language model. So what is a large language model? So first of all, a large language model is a language model. So a language model is a model that assigns probability to sequences of tokens. So in this case, our model always predicts the probability of the next token. So it's, in that sense, a language model. But also, a large language model is large. So why is it large? So we're going to see that these models, they are actually scaled up in terms of size. So first of all in terms of model size. So these days, it's not uncommon to see models on the order of hundreds of billions of parameters. But typically, when we say LLM we say at least on the order of a billion. These models, they've also been trained on a huge amount of data and hereby amount of data, we quantify that by the number of tokens that they were pre-trained with. And this is on the order of magnitude of hundreds of billions of tokens or even trillions of tokens. So I think the biggest ones are like on the tens of trillions of tokens. So huge training sets. And they're also large because they need a lot of compute. So typically you need a bunch of GPUs to make them work. Although these days there's been a lot of optimizations to have them work on consumer based GPUs. So we're going to see that later on. But in LLM is large according to these categories. So another thing that I want to point out is that all these terminologies, they're relatively new. So I remember in 2018, '19, there was no real definition of an LLM. I think no one actually talked about LLMs. In the beginning, maybe people talked about LLMs. They included BERT. But Bert is an encoder-only model that does not produce text. So with the current definition of an LLM, which right now has been pretty well established, BERT would not be an LLM because it doesn't produce text. So here, we only consider language models that do text to text, that are very large in size, in terms of amount of data that have been trained on and in terms of compute. Cool. And as we saw before, these models are decoder-only. So here, what we do is we remove the encoder, we only keep the masked self-attention, the feedforward neural network, and then the addition and normalization. So we only keep this. And this is the backbone of LLMs. And so I mentioned GPT is kind of a good example. But it's not just that. You have plenty of other models. So you may have heard of LLaMA from Meta, Gemma from Google, DeepSeek, Mistral, Qwen, and so on. The list is long. I would say roughly-- I mean, more than 90% of modern day LLMs, they're all decoder-only. So I think that's something to keep in mind. Cool. OK. So now you how LLMs are made of.

Mixture of Experts

But there is something else that people these days also introduce to these models. And we're going to see that in a bit. So I mentioned that these models are huge in size, again, typically hundreds of billions of parameters. So it takes a lot of compute to just compute one inference, or also to train these models. But you may wonder, do you really need to have all these parameters be activated during a forward pass, to make a simple prediction? So I'm going to have a little metaphor. So let's suppose you enter in a room. And in the room, there is a mathematician, a physicist, a chemist and a historian. So you come in this class, there's a bunch of people who are expert in what they do. And you have a question, you have a math question. So a question I have for you is, who would you ask your question? Would you ask the mathematician? Would you ask the chemist? Would you ask everyone? Well, right now, we ask everyone. We ask all parameters of the model to be involved in the computation of the generation. And so the idea here is given an input, maybe it's not necessary to ask everyone to be involved in the computation. So the idea is let's actually just have a subset of the model be involved in the computation of the next token. So I'm just introducing this idea of experts. So let's suppose we're introducing the following notation. So let's suppose we have n experts. So think of it as your mathematician, your chemist, historian, whatever. So these are your experts. And the idea is given an input x, you're going to ask yourself who should be involved in the generation of the outputs. So you're going to have, let's say, another network. Where we're going to call it G like gates, but it's also sometimes called router. So let's suppose we have some gates that tells us which experts should be involved in the inference. So if we have that, so let's suppose here, the gate tells us OK, so actually, expert number 2 is well suited to answer your question. So here, the idea is that the input is just going to flow into that expert, but not the other experts. So this has a name. It's called mixture of experts. So it's denoted MoE. Everyone talks about MoE. So these are a mixture of experts. And so the formula that you will see a lot is this one. So the output y, which is denoted y hat is the sum of the expert output weighted by some quantity, which is the output of the gate, which tells you how important the output of each expert is. Yeah. Great question. So the question is, how do you train G? How do you train E. So typically, you train them jointly. So we're going to see that maybe in a little later. But you can think of it as just training as usual. You do your forward pass, you compute the loss and you backprop. And it's actually an interesting question because there is some challenges that come with training MoEs that we're going to see in a second. Yeah. Question is, what are these E's. What is the architecture of these E's? So let's suppose right now that there's just some network. We're not specifying them for now, but we're going to see this in a second. Let's suppose for now, it's some network. Cool. OK. So I told you that what if

we don't activate everyone. What if we activate a subset? But this formula, actually, I guess, assumes that we're actually considering all expert outputs. So I just want to distinguish two kinds of MoEs.

Dense & Sparse MoE

So there's one kind that is called a dense MoE. So a dense MoE actually does not have any constraints on the number of experts that are involved. So these weights, they can be anywhere between 0 and 1. So think of them as a probability distribution. But it's just going to put more weight towards some experts compared to others. So back to the example that I had, so let's suppose I have a math question. So I'm going to ask the mathematician, the chemist, the historian. So I'm probably going to add a higher weight to what the mathematician says compared to let's say, the historian. So this is the idea. But then the interesting thing is when we constrain the number of experts that are activated, because here, as we mentioned previously, what we're interested in is to not involve everyone is to make some savings in the amount of compute that we do. So there's a second kind of MoE that's called the sparse MoE. And what it does is it only selects the quote unquote, "top k" experts. So k can be equal to 1. So one expert or even two. So it's a hyperparameter that you choose. And so here, the expression of the output becomes the sum over all the chosen experts of a G of x times E of x. So far, so good? Cool. And of course, there's a lot more to it. So in case you are interested in learning more, feel free to go into the resources that are at the bottom of the slide. So one thing that I will say is that we have a unit of measure of the amount of compute that these models produce for each pass. So you will see the term FLOPS. So have you seen the term FLOPS out there? No, not really. So it stands for floating-point operations. So it quantifies how many operations. Think of it as additions, multiplications are involved in a forward pass let's say. And it basically quantifies how compute heavy is your task. So typically, what we say is when we go through a sparse MoE as opposed to a dense Moe, we have a lower amount of FLOPS. So this is the unit of measure that you will see.

MoE in LLMs

But back to your question, so what are these experts? So if you remember, I mean, 10 minutes ago, we said that LLMs, they are decoder-only models. So I have a question for you. Let's suppose we wanted to put some MoEs in our LLM. Where would we put it? So here, I guess, we have three choices. We have the masked self-attention layer. We have the feedforward neural network. And then we have this normalization. So question for you, I guess, where would you put this? I guess, where do you think is-- I guess, the most complex parts of the network. Where is there a lot of operations?

Feedforward? Yes. Yeah. Great answer. So it's indeed the feedforward neural network. And the reason for that is I think Shervin mentioned it, I think, in lecture 1. So if you remember, the feedforward neural network is a network such that you have the input, which is your D-dimensional input vector. And then you have this being projected into, let's say, a DFF dimensional space. And then it goes back to the D-dimensional, I guess, space. So the DFF is typically larger than your dimension of, I guess, the inputs. When I say input, it's here. So it's typically larger. So the amount of parameters that you have in that feedforward neural network is something on the order of magnitude of D model times DFF times 2 plus some bias. So it's basically your order of magnitude. And the attention layer, if you remember, it's basically composed of the projection matrices, what is the dimension of the projection matrices. So it's D model times the dimension of keys to dimension of queries the dimension of values. And this dimension is typically much lower. So think of it over 100. So your D model is typically O over 100, O over 1,000. And then the projection here, the DFF is O over 1,000, O over 10,000. Cool. So is everyone now convinced that this is a good way, good place to put the mixture of experts? Yeah? Cool. So this is actually how it's done. So in modern day LLMs, this idea of not involving everyone in the computation of the next token prediction is such that you would put the mixture of experts where the fn is just basically here. And typically you would have a sparse mixture of experts. So back to your question, so these experts are actually feedforward neural network. So you would have several networks that you can train, but you would only activate one. So typically, k would be equal to 1. It can be also equal to 2 but it would only be a subset. So that's my point. And this routing would be done at the token level. So if you remember, the decoder, it basically takes something as inputs. So a bunch of tokens. And here, what I'm saying is that each token will be processed by an expert that may be different from the other token. So the router here would take the representation of the token as input and figure out which expert should be best for this token to flow towards. Does this idea make sense? So I have a little illustration later on that hopefully will help. So now back to your question about how you train this model. Do you train the router separately? Do you train the experts separately? So one challenge that people have is when they train these MoE-based models, to make sure that all experts are, I guess, having a weight, are being used. Because it's very possible that you train your model and that somehow only, I don't know, one or two experts always get activated. And the other one, they always are inactive. They are never involved in the computation. So this problem is called routing collapse. So why is it called routing collapse? It's because the router always chooses some experts, but not others. So this is a challenge. And the way people try to mitigate this challenge is by changing the loss function and adding it some extra term, which is written here. So it's basically some hyperparameter alpha times the number of experts times the sum of quantities that

depend on whether or not tokens went to a certain expert's eye, and then summed over all experts. So it's not super important that you completely understand exactly how that formula works. The only thing that I think you should take away from this slide is that this extra loss allows these quantities to converge more towards uniform distributions. So what are these quantities, just as a reminder. So f of I is the fraction of tokens that are routed to expert I. And P of I is the average routing probability for expert I. So when I say that all experts should be used the same, what I'm saying is I want this probability to be kind of uniform across experts. Yeah. So the question is, I guess, when do you compute these quantities? So yeah, you can think of it as a regular training process. You do some mini batch, you go through the model and then you compute all these quantities. And then what you do is you do your backpropagation based on that. And I guess what I want you to remember is that this incentivizes the probability, I guess, the choice of the writer to be more uniform across experts, which is something that mitigates this routing collapse phenomenon. Yeah. So the question is, can we use Dropout? Of course. You can always bundle that with some other techniques. So people have just kind of found this to be very helpful. So speaking of other techniques, there is something that I've not talked about, which is very similar to the Dropout idea. So it's called noisy gating. Noisy gating is basically you have your predictions from the gates, but then you add some noise to it. So basically, it's by pure chance, it just allows other experts to be involved in the computation. So it's also some other technique. There's a bunch of techniques. But yeah, Dropout is indeed quite useful for things like overfitting. And the idea can be reused in different settings. Yep. Yep. So the question is, how can you-- so do you mean differentiable? So here, I guess, how do you take the derivative. Is that your question? How is the derivative-- can you explain a bit more what your concern is? So I guess, to this question, so the average routing probability. So that one is a function of the gate output. That's one. PI. So I guess, your question is for FI. For FI, OK. I don't have a good answer on top of my head, but I think people have some techniques. And these days, you just don't even have to do this by hand. You have the built in thing. So maybe I can follow up with you for FI. But for P of I, do you see that this one is quite clean. It's just the average of the probabilities from the gates. Yeah. So basically the probability-- the output probability from the gate, you can think of it as just the vector being projected on a space of n, where n corresponds to your number of experts. And then it's gone through Softmax. So your output is basically summing up to one. And each of these dimensions, they represent the value corresponding to what expert I would be, I guess, used for. For instance, the first dimension would be for expert 1, second dimension would be for expert 2, and so on. So you just take the average of this, and this one, you can express it from all the parameters. So I think you should be fine with that one. Yeah. So the question is if we increase the number of MoEs, does it

increase the number of model parameters? So it's a great question. So it's actually one of the ideas behind MoE-based models, which is that you can scale the model without having to incur the cost of having significantly more compute at inference time. So you can increase-- so people say capacity. Can increase the capacity of your model, but you will still keep some, I guess, like controlled amount of active parameters. And active parameters are the parameters that are used for a forward pass. So yeah, people just use that. So yeah, it would just increase the number of parameters. So that's why you see some MoE-based models that are even bigger than the ones that we had on the order of hundreds of billions. We even have on the order of trillions of parameters. So for instance here, one reading I recommend is switch transformer, which scaled up to one point something trillion parameters. So yeah, it's definitely more. But that being said, so if you read the paper, you will also see that these models, there are more what they call sample efficient. So they take less time to be as good as what the model would have been with a lower number of parameters. So if you draw the training curve as a function of the training time, you see that these models are typically more sample efficient. Sorry. Yeah. Yeah, exactly. Everything here is a trade off. Everything here is a trade off. Yeah. Cool. Yeah. So the question is, each attention head will have a number of experts. So it's actually regardless of the attention heads. So the attention heads, you can think of them as being independent, something else. And the number of experts is independent of that. Does that make sense? Yeah. Right. All right. Yeah. The question is whether every block will have a number. The answer is yes. And typically those weights are not shared. So typically, you-- so actually, we're going to see an example. It can very well be that layer 1, there is the expert number at all. Three that was chosen, but layer 2, there is expert number 1, and, you know, it's all free. It's trainable. So the question is we will decide where the expert will go to. So all of that is decided by the gates, which is this quantity. Everything is decided by the gates, which has trainable weights. So you can think of this as just some projection from the input x to an n dimensional space, where n is the number of experts. So the question is, at what point during the inference is it decided? So let's suppose we're at inference time. I'm going to walk you through how it works. So you have your x. So you have this attention mechanism. So it interacts with all tokens from the past given that its decoder-only. So it's like the mask. It goes here. And at the beginning of the feedforward neural network block, the token is, of course, contextual. So it has the information from these other tokens because it's attended. And what it does is it goes here. So x first goes into G. G computes this probability distribution over all experts. And given here that we are in a sparse MoE setting, we will only choose the top k. So let's suppose the top one, just the highest probability. And we know which experts this one will be. And so as a result of that, you will only compute the output value of the expert that the input was chosen. So can you elaborate on that actually? Yeah. At what

point exactly? So it's after the self-attention layer. Yeah. So the question is, do we have different classification for different heads? No. So there's only one router. So I think I understand your question. So your question is, what do you do given that you have different attention computations going on in parallel with the heads? So if you remember, the attention layer has these different heads. But at the end of it, what it does is it concatenates all the results from each of these heads and then projects it once again in the D model space. Yeah. Yeah. Yeah. Yeah. So the question is, do we have different Gs? So the only thing I can tell you is that the G is just layer specific. It's layer specific. It's trainable. So it's basically going to learn how to process all these inputs. So the G, the only thing I can tell you for your question is going to be layer specific. So the G is going to be one G for let's say the first layer, another G for the second layer, and so on. And that one is going to be trained. Cool. Great. Looking at the time. Do we have any other questions here? We're good. Perfect. So now I just wanted to show you a cool thing that I believe the Mistral team was showing in one of their papers. So what they're showing here was for a given piece of text, to show in which experts each token was rooted. So as we noted before, so experts are different from one layer to another. So I believe here, for layer 0, so it's like for one given layer. And we do see that roughly, these tokens, I guess, they leverage like a uniform amount of experts, more or less. What you would not want to see is to have every token be the same color. But luckily it is not. But yeah, so that's one cool way of just representing how the routing is done is to just have your input text and just represent where each token-- in which experts each token went. Cool. OK. So what we just saw was one way that modern day LLMs changed their architecture to incorporate the fact that we may want to scale the model, but not increase the computation complexity for one forward pass. And we saw that with MoEs. So you will see a lot of MoE-based LLMs out there. And now what we will do is knowing that we have an LLM, we're going to focus on-- don't worry. We're going to focus on how a response is being generated.

Response generation

So remember, when I told you that these modern day LLMs, what they do is they take some text in and they have some text out. So it's typically these task of next token prediction. So you have a token in. So let's say, beginning of sentence, you go through your LLM and it just generates the next word or the next token. So A, and then you take A, and then it goes to teddy. And then teddy bear is, et cetera, et cetera. But so far, we have never really dug into exactly how we chose the next token. So what we're going to do right now is to see exactly how we're generating the next token. So as you know, here, our LLM is just a decoder-only architecture. So here, what you have is a decoder with your inputs here, and then your output there. So let's suppose for a

second that everything that's happening in the middle. And we're just obtaining output probabilities that are going to look a little bit like this. So given a token or some sequence of tokens as inputs, you have an output probability distribution that represents what the model thinks is the likelihood that there will be a next token, let's say, that is equal to A, to airplane, to fluffy, et cetera. So this is what you have. So now my question to you is, if we told you we have some sequence as inputs and we want to choose the next token, and if I told you that our model is giving out actually a probability distribution, I guess, how would you choose the next token based on this?

Greedy decoding & beam search

Sorry. Great. The token with maximum probability. OK, great. So yes, the first idea. Let's just take the token with highest probability. So it's a very natural approach. But I'm not sure if you've been using things like ChatGPT or Gemini. Every time you ask something, it always responds something that is slightly different. So if you always choose the token with the highest probability, given that the computation here, we're going to see is all deterministic. What that means is you're always going to generate the same thing regardless of, I guess, with the same input. So that's one limitation. So it's not very diverse. The second problem is if you choose the highest probability token on, I guess, iterative basis, your locally optimal, but you're not necessarily globally optimal. So what does that mean? So I guess, if you think about it, our objective is for us to produce a sequence, an output sequence of tokens that is, I guess, of a high probability. But the problem is, if you always choose the highest probability token, you will not necessarily obtain the highest probability sequence. Are you convinced of this statement, by the way? So let me give you an example. So let's suppose you have the next token where one token is 0.8, the other one is 0.7. And then you choose to go with the 0.8. No, actually, it's not 0.7 because it has to sum to 1. So let's say 0.2. So let's suppose if you go ahead with the sequence that starts with the 0.8, let's suppose all other token probabilities are very low. Basically, you will have an output sequence that will have a lower probability than let's say, the other path, which would, let's suppose, have higher probability predictions in the later steps. So we're going to see this in a second. But this is the idea. So if you choose the highest predicted probability, it's a good first idea. But it's locally optimal, but not necessarily globally optimal. And this is the reason why we have a second method that is about keeping track of the k most probable path. So I'm not sure if you've heard of beam search. So that's what beam search does. So here, k, is sometimes called the beam size or the beam width. So if you hear these terms, these are just names that are given to the number of paths that we keep track of. And so this works as follows. So let's suppose we start our generation with the beginning of sentence token. We want to

figure out what the next token is. So let's suppose we have here, in this example, a very basic example like three tokens. And let's suppose the two highest probable tokens are A and the. So if we have k equal to 2, what we're going to do is to keep track of these two branches. So that's the first iteration. The second iteration is we're going to look at all the probabilities of next token prediction for these two tokens. And we're going to always save the two most probable paths. And here, for instance, let's suppose if it's like the, and then fluffy and then A and cute. So back to what I was saying earlier, so what I was saying was here, if you were choosing the path that went along the highest probability token path, like the the, it's very much possible that the highest probability token after there would be a much lower probability than the one after A. And so this is what beam search tries to do. It tries to have a more globally optimal solution. So let's suppose we continue that. And then at the end of the day, we obtain, I guess, a number of potential, I guess, choices and the k potential choices. And then we pick the one that is the highest, likely, the sequence with the highest probability. So people typically what they do is they take the sum of the logarithm of the probabilities of the tokens. So what they say is they say, OK. So the log probability of the sequence is the sum of the log probability of each next token prediction. So it's the log probability of A, knowing BOS, and then cute, knowing BOS and A, and so on and so forth. But I just want to point out one limitation of this approach, which is that the more you generate tokens, the lower your, I guess, end sequence probability will be. Because if you think about it, all these probabilities are between 0 and 1. So think of it like in the multiplied sense. So let's suppose you have the probability of the whole sequence, which is just a multiplication of the probability of the next token. The more you add probabilities below, like less than 1, the more this quantity will go towards 0. So I guess this method, as is, will prioritize sequences that are shorter. And so for that reason, beam search has some additional term that basically counteracts that effects. So something on the order of 1 over number of tokens to the power of something. So in practice, there is some technique to make sure that things kind of work relatively well. OK. Let's suppose we figure out all these things. The problem is that we need to keep track of this most probable path. We need to do all this kind of saving, et cetera. And it requires a lot of computation. And the other thing is we're still interested in the most probable path, which basically will lead to a sequence that is very-- that the model thinks is very likely. But sometimes what you want is for your output to be more diverse or more creative. So that's why beam search is actually not something that people typically use. People use beam search for things like machine translation, where you actually need to have something that is close to something being very likely. But actually, people use a third method.

Sampling-based methods

And this method is called also the sampling method. So I told you we have a probability distribution over tokens regarding what the next token should be. And so what people do is they just sample the next token using that probability distribution. Does this make sense? So in this example, fluffy, gentle, kind, and let's say smart will have a higher probability of being drawn as opposed to let's say airplane, that have a lower probability of occurring. Non-zero probability, but they have a probability of occurring. Cool? Any questions so far? Yeah. Right. So the question is it's not for training, it's for inference. Yes, correct. Think of it as a response generation. So let's suppose you have your model that is trained. What you want is to generate an output. So what would you do? Yeah. So I guess, just to complete my answer, so during training, what you would do is care about the output probabilities and then compare them with the actual label, which is, most of the time, like your hard label. And yeah, this is what you would compare. So this one is let's suppose you have an LLM that is trained, how would you generate a response? Cool. Are on goods? Yeah. Yeah. So the question is, how do you do sampling in this situation. So it's actually my next slides. But I just want to make sure everyone was on the same page regarding just the intuition. So highest probability is called greedy decoding. It's probably not something that we want. Beam search is a little bit better. It's more something that is globally optimal. It's not globally optimal but more towards that. So it's better but it lacks diversity. It lacks creativity. Which is why what we want to do is to actually sample each token. And I guess, we have a few methods that also restrict the kinds of tokens that we want to sample from. Because I mentioned, these very low probability tokens, they can still, theoretically, be sampled. But it's not necessarily something that we want. So what people do is they typically restrict the highest probability tokens and only sample from them. So you may have heard of this term, Top-k sampling. Who has heard of this term? Yeah, a little bit. So what you do is you select the top highest probable k-- so the top k highest probable tokens and you sample from them. So let's suppose if k is equal to 4, you take the four highest probable. And then you just sample with them. And there's something else that is quite similar in idea which is called Top-p. So Top-p is that you limit yourself to the top to the highest probable tokens such that their cumulative probability is more than a threshold p. So again, it will do the same thing. It will select the highest probable tokens. But then there is a part that I still left out, which is how do you obtain these probabilities to start with? So if you remember, here, I had mentioned that we just assume we have the probabilities. We just have them and we want to choose what the next token is. But now the question that I'm asking is now that we know what we will do with these probabilities, I guess, one question is, how do you obtain these probabilities to start with. So if you

remember, these transformer-based architecture basically computes an encoder representation of the input. And what you have is some final layers at the very top of the figure, which aims at projecting the vector into the space of the vocabulary. Because what you want to do is to have a probability number for how probable it is for you to sample a given token. So here, what you would do is at the very top of the architecture, have your input, which is your encoded embedding of the token. Go through a linear layer, which basically projects your D model vector into a space of dimension size of v. And of course, what you want is probabilities. So you would have a Softmax layer, which basically converts everything to probabilities such that everything sums to 1.

Impact of temperature on predictions

And in order to compute the probabilities, this is the formula that you would use, which is the Softmax layer. So there is a hyperparameter that is quite important that I want to talk to you about, which is the temperature T. And so this is where it pops up. So you have the probability of the next token being a given word, which is equal to the exponential of the input with respect to the given word over temperature. And then you normalize that by the sum of all the other exponential of the quantity over t. And now what we're going to see is what that T is used for or what that T actually does in practice. So before we go into that, just one question. Has anyone heard about temperatures when it comes to response generation? Yeah. Cool. So hopefully that will give you a better idea of how the temperature kind of influences your output predictions. So I guess, the question that I want to ask you is, what would be the impact of having a low temperature versus a high temperature? So low temperature would correspond to a diverse of what? Yeah. Yeah. So we will come to that in a second. So yeah, do you have a-- a lower temperature? Increase in temperature. So I guess, back to the explanation about, I guess, lowering the temperature and what impact it will do to the-- so I guess do you have a suggested response to that? I guess, lower temperature. Let's suppose you have a lower temperature, what happens? Yeah. So I guess-- so the long story short is low temperature will create a spiky distribution and then high temperature will create a uniform distribution. But I guess, I think you had the right intuition. I'm going to maybe write this in more mathematical terms. OK. Let's suppose you have your probability, which is the exponential of xi over t, over the sum of exponential of xj over t. So just mathematically, you can actually prove what is happening here. So let's suppose you have the index of the highest xi. So let's call that k. So let's suppose what I do is I will factor by this quantity. So actually, here, I don't have space. So let's do here. So it will be ae of k temperature. And then here, you have the sum of over all j of exponential of xj over t minus xk over t. So what I did is I just

multiplied the numerator and the denominator by exponential of xk over t. So I multiplied numerator and denominator. And of course, they cancel out. But then what I have here is xy minus xk over t. And here, xj minus k over t. So when i is equal to k, this term is 0. It's exactly 0, regardless of the temperature. And here, xj minus xk is always negative or 0. So here, if i is equal to k, you have 0 over t. So here, it's 1. And then you have 1 plus-- I guess, some quantity. So j minus xk over t, which is going to be negative. So the t going to 0. So that one will be, I guess, going to, let's say, minus infinity. So exponential of minus infinity is 0. So what you will end up with is something where for i equal to k, you will have a non-zero probability. But then for i not equal to k, you will have 0 over something that is not equal to 0, which is 0. So just mathematically, if you factor by exponential of xk over t, where k is the index of the highest value, I guess, the highest logit or the highest activation vector, can actually show that for a small temperature, only the highest-- so the index of the, I guess, highest value will be the one that will be highest probable with the spike there. And then when you have a high temperature, basically it will look like a uniform distribution because all these quantities, so when t tends to plus infinity, this tends to 0. So exponential of 0 is just e. Sorry, not e, 1. So 1. So it's 1 over the number of j. So it's 1. So just a uniform distribution across all the tokens of your vocabulary. Yeah. So the question is, how can I interpret a small-- so the question is, can I interpret that as a Gaussian? I think. So it's kind of tough because Gaussian, you have some continuous quantities in the x-axis. So here, you have tokens. So it's kind of discrete. And it's not something that you can order. So I just probably interpret that as small temperature is very spiky. So you will really have a given set of tokens that are the highest probable that will appear more. And if you have a high temperature, then basically, tokens, even the ones that were not there to be the highest probable, will actually have an adjusted probability that will be higher. Yes. So you can think of it as some kind of scaling in that sense. So long story short, what I want to tell you is if you have a small temperature, it will encourage the next token to be geared towards being the highest probable token. And then if you have a high temperature, I guess the distribution of probabilities will be kind of closer to a uniform if you're really increase that temperature by a lot. So your output will be more creative. You will have tokens that you would have not drawn that would be drawn. And so what that means in practice, if you're with your favorite LLM. And let's suppose you want to, I don't know, write something very creative. So which one would you use? Would you use a low temperature or high temperature? High temperature? Yes. So if you want something that is more deterministic, more, I guess, closer to something that is very, I guess, what you think would be high quality, will use more of a lower temperature. So one note here is that if you have a strictly positive temperature, every time you run your model over an input, you will obtain a different output. So something I want to point out is nothing in the transformer architecture is

probabilistic. Everything is deterministic. The only thing that is not deterministic is how you sample the next token. It's the only thing that is not deterministic. So what will you do to have a deterministic output? You have t equal to 0. T equal to 0, you know for sure that your output will be just one thing and nothing else. Well, that's what it's the theoretical property. But in practice, you have some stuff that is happening in the computation that introduces some non-deterministic operations. It's much more advanced than the scope of this class, much more advanced of the scope of this class. I just want to call out that in practice, t equal to 0, may lead to different results because of this kind of practical piece. So what I recommend. So there's a suggested reading in case you're completely optional. So there is a very recent article around defeating non-determinism in LM inference. So I recommend that you read it in case you're interested. So the high level idea is our GPUs, our hardware, when they reduce some of these operations, sometimes they are reducing numbers that are on really different scales. So the order at which these operations are happening is actually quite important. So the idea is if these operations they happen in different order, it may lead to different results. Even if in theory, it should be exactly the same, in practice, it may not be the same. And this article actually does a great job at just explaining the intuition. So yeah, just if you're interested, completely optional. Cool.

Guided decoding

I know I'm kind of late, so I guess, last thing that I want to say is let's suppose, what you want to do is to generate an output in a very specific format. Let's suppose a JSON format. So I guess, a very naive approach would be to tell your LLM, well, please generate this in JSON. And then it produces something. And then you try to see if it's a valid JSON. If it's not, you repeat-- you just tell it to generate again until it does something well. So that's the first naive approach. Well, there is a second approach that is called guided decoding. And what that does is during the generation process, it filters out what it calls invalid next tokens. So here, let's suppose I want to generate this JSON. I know for sure that my first token must be something that opens the-- I forgot what the English word was for this, but it just opens the brackets. Yeah. Open the brackets. So you can only have this and then you can only have the property name and so on and so forth. And sometimes you can have more than one permitted next token, in which case you would go back to our next token strategy. Yeah. So the question is, how do you restrict the other tokens? So there's a bunch of papers that go into that. We will not cover this. But I can give you some pointers. Just type finite state machine FSM context grammar. So there's a bunch of papers that do that. We will not cover that here. Cool. And I think with that, we will go to the second part with Shervin. Great. Thank you, Afshin.

Prompting strategies

So now, together, we're going to look at different prompting strategies. So now that how responses are generated, we're going to see how to get responses and how to get great responses. So let's go back to our favorite example about our cute teddy bear. So just I want to introduce one piece of vocabulary. So when you have some kind of input, you have the length of your inputs measured in number of tokens. And you will see in the literature that it can have different names. So you can see it called context length, context size , window size. And I think the latest tools if you code on Cursor or any other code-assisted tool, I think they call it more context length, but all these terms are equivalent and denote the same thing. OK. So now I want to take some time to discuss what can be some orders of magnitude here. So like modern day LLMs, they tend to be in the order of magnitude of tens of thousands, hundreds of thousands or millions of input tokens as inputs. So this is the kind of input it can take. So sometimes, you hear some new LLM boosts some number. And then that number of context length is exactly that. So it's how much tokens can it accommodate in a single pass. OK. Great. And then yeah, the models, you have seen maybe some advertisement of these models. So I think at Gemini and others that are in the million territory. Does that mean it's all nice and pretty? If we have more context length, is it going to solve everything? Actually not. There is a phenomenon that was recently dubbed as context rot in a paper from earlier this summer, where basically, the authors are experimenting the capacity of the model to retrieve some piece of information in a test called needle in a Haystack. So basically, they ask some question. And in larger and larger pieces of text, they bury the answer. And they try to look at the capacity of the model to surface that answer. And then you can see that with respect to an increasing context length, that ability to ground the information with the answer that is in the context decreases. And the paper I think is a very interesting one. It controls with respect to what else is in the context. There is a term of distractors, where they put some noise and then they see distractors basically contribute in decreasing the retrieval capability. So also, what is in your context matters. So this is why in general, when you have a retrieval problem, where you want to use your LLM to solve it, you have some good incentive to try to target as much as possible, the right piece of context for your LLM to see and predict on. OK. Great. Yep. Yep. Yeah. So the question is, is the context length exactly what is shown during self-attention? Yes. Yeah. And then we're going to see-- so I think we saw last time that sometimes there are some tricks that are used for the complexity of the self-attention mechanism not to be n squared. So this is what is being used to manage basically the complexity of the computations as the context length grows. Yeah. But great call out to that. That's exactly what is it about. OK. Great. Any questions? Any other questions? OK. Great. So

now, let's define what do prompts usually look like. So there is no formal theory on how to structure a given prompt. But these are the main lines that are usually found in prompts presented to models. So you can distinguish a part that is setting up the context where you just put the setting, you make it clear to the model, what kind of query you want to issue. Then let's say, if you have some tasks that you want the model to perform, you have instructions. And then these instructions are some of them function that take as input something. So you add some inputs. And in order to get what you want out of the model's response, you may want to add some constraints out of it. So here, we took our favorite example with a teddy bear. The context is what is the teddy bear-- basically, what is the mood of the teddy bear? What do we want to do in order to make our teddy bear happy? And then we give some input regarding where the story that we want to generate is. And then we give some constraints. If you want some other example in the LLMs that you query every day, so the context could be some sentence that says, you are ChatGPT. We are October 10. It is 4:44 PM. So you set the setup. The instructions can be basically what the user queries alongside the inputs. And then constraints might be things that you don't see as a user, but that might be present in the prompts given to the LLM, such as safety instructions. For example, do not generate any contents that might lead to harm, or things like this. So every time you see an input being fed to an LLM, I think these projections, these four dimensions can be a good mental model of seeing which pieces correspond to what goal in that structure. Does that make sense? OK, great. So now that we know what kind of mindsets

In-context learning

we might have for inputs to an LLM, let's see how we can get an LLM to do what we want. And let's see how we can get an LLM to do what we want without tuning any weights. And we can do so with a concept called in-context learning, where the learning is a bit of a term that is overloaded because you don't actually learn anything with respect to the weights of the LLM, but you can distill some knowledge inside of it as part of the context in order to do what you want. So you distinguish two main categories of in-context learning. So there is one category that is very simple. In mindset, it's called zero-shot. It's basically you don't give anything other than your input, your input query. You just ask the LLM to do what you want to do. And then there is another school of thought called few-shot learning, where you give examples of inputs and outputs to the LLM before asking the input that you're interested in. So in the case of a teddy bears and a bedtime stories, so you might name your teddy bears. So maybe you have teddy bear that's called Teddy. And you generated a story. So you put your query, you put the story that you want to generate, and you give multiple such

examples. And let's say you have another teddy bear that's called Bob. And then you want to generate a story for Bob. So you put all of these examples, and you ask the LLM to generate the story that you're interested in. And then this is basically what we call a few-shots setting. OK. Great. So now, generally, when you look at the performance of the resulting task, giving examples tends to steer the LLM nicely into the task that you're interested in. Just because you have given an idea to the model of what you are looking for. And then it can use what it has learned during its training process to connect the dots and basically replicates. But of course, you need to gather such examples. So this is costly. This will put some more tokens in the context window. So you will need to more compute. But there is an interesting trade off here that we see in recent models. So they are gaining more and more reasoning capabilities, which is why I put the generally nuance in italic here. So they are generally better, few-shots, but not always. Because these days, with these better models, people have seen that with basically making the instruction better, you could make the performance of in-context learning on par, or even better than if you provide examples, because basically, when you think of it, when you provide examples, you constrain your model into given sets of examples that are finite. So when you are at inference time, let's say, you want to perform a task on a distribution of data that has not been seen, it is harder for the model to generalize because it will try to align on what it has seen in the context. Whereas if you turn your instructions into something that is more reasoning based, you explain how to do the task with natural language, it can use its reasoning abilities to actually do it. And this is something that you see more and more these days. So the literature on this, I think, is still in progress, but you have some papers like a plan and solve that's I think came out recently. And that shows if you ask the LLM to plan itself and then solve something, it can have very good performance. So yeah, this is an interesting fact.

Chain-of-thought, self-consistency

OK. Great. So now that we have seen basically the kinds of learning that we have, now, we're going to see what we can do to improve the quality of the response. So there is this concept called chain of thought, where researchers have seen that if you force the model to come up with some rationale to some answer before actually giving it, it would give higher performance. So this is what people call as chain of thought. It's basically what led you to the answer. So here, for example, if you ask yourself, how old is this teddy bear? And if you force the model to respond directly with a number, it might not be able to do the connection as to exactly why the Teddy bear is a given age. Whereas when you give the full chain of reasoning, then it becomes clear as to what led to that response. So I think this is the main mindset behind this technique.

And basically, this is something that can be used in in-context learning. So if you give few-shot examples, how old was this Teddy bear, you give some number and then you slightly change the query. And then based on the reasoning, formatting, and then the response, the LLM can adjust the reasoning and the response accordingly. And it will force the model to output some reasoning alongside the response, which shows an improvement in basically metrics. OK, great. And then yeah, something else that I want to say, let's say, you want to do a task and you want to do it well. m you will always have some samples that do not work. And then for them, you want to have debugging ability. And usually when you debug with LLMs, you don't debug like the old days. You don't look at weights at the intrinsics of the LLM. What you want is something more interpretable that comes out of it in terms of tokens. And usually, with these techniques, you can see the reasoning that the model had before outputting a given response. And you can debug basically what went wrong. So if you ask how old will the bear be next year. And if in the chain of thought it says, hey, we're in 2019, you know that somehow in your context, oh, maybe I put the wrong dates in there. So you can trace back to-- you can do some root causing very easily. So it's also used for that. And, exactly, in the same way as other techniques that increase the number of tokens, this is a trade off that you have to manage, that it will, of course, take more time to infer because you need to generate more tokens. But typically, we're fine with it. Any questions on COT? OK, great. So now let's take it one step further. So I want to introduce even to make COT even stronger, you can sample your model. Several times, look at the answers it generates, and then do some majority voting to select the answer that was predicted most of the times by a given LLM. And this is what we call self-consistency, where basically, you sample responses multiple times and then parse the actual answer. So basically, you factor out the whole reasoning and then try to target the answer. And then by majority voting, you can come up to an answer in a more robust way. Yep. Yep. So the question is, do you need to have some benchmark to know if it's better? And yes, absolutely. I think the paper operates on top of all the arithmetic and then mathematical based ones. And I think one tangential question to yours is how can you locate the answer and then do measure majority voting on it. So typically, you can put into the context some instruction that says, put the actual answer last. So you can just extract the last. People, I think, use regex based techniques, otherwise, to extract the answer. Or you could even think of using some other LLM to extract the answer. So there is always a way to do it. But yeah, to your point, yes, we need some benchmark and then ground truth labels to assert that this is indeed better. Yeah. So the request here-- so the question is, is it going to be sent to the same request as example? So typically, you sample it in parallel. So basically, you ask your model exactly what Afshin just mentioned. You sample from the probability distribution of BOS and whatever tokens you had, the next token. And you do that in

several other kind of branches. And you do that, all of that, in parallel. And then you parse the answer at the end of it. So basically, the latency of this process is equal, roughly, to the maximum latency of either parallel generation. Because you can do all of this in parallel. So none of these go into the context of each other. They're all done in parallel. And then you get the final answer. OK, great. All good? OK, great. So now that we have seen a few prompting techniques, I want to cover together tricks that people have come up with at inference time to make the generation as

Inference optimizations with KV cache

efficient as possible. And basically, yeah, so in modern day models, you have a lot of parameters. You want to generate a lot of tokens. How do you do it the most efficiently possible? So I want us to divide this thought process into two parts. So we're going to see what methods we can come up with that give us increased efficiency on an exact level. So you exactly do the same computations as you were supposed to do, but in a more efficient way. And I put some hints as to what kind of techniques we're going to explore. So let's try to look for techniques that avoid redundancies. Let's try to see what we could do to manage the memory in the best way. And also, let's try to see if we can reformulate the equations inside the LLM in a way that could simplify the generation in practice. Then there is a second category that we're going to explore as well. That is going to focus on what kind of approximations it might be fine to do, and get a high quality answer at a lesser cost, but to be approximate. So we have what kind of variations we could do on the architecture? Is there something we could do on the embeddings to make them more efficient? And then maybe on the token prediction side, is there something we could do to make it faster? Does that sound good? So we have approximately 22 minutes. We're going to try to go through all of them here. OK. So I grouped the categories of techniques into exact and approximate techniques. But actually, we're going to look at them in a grouped manner. First, looking at the attention layer, seeing what kind of techniques we could have there. And then in a second part, we're going to look at the output layer. And then see what we can do at the level of token generation to make things as good as possible. So let's start at the attention level. So we have seen that for every token, you need the current token to attend to the previous ones in this masked self-attention. So let's say you generate your sequence. You are at a given token. So let's say, you have generated a cute teddy bear. And then now you are at ease. Now you have the query ease. You will compute the key representation and the value representation of ease. Because of course, you have to do it. It's your current token. But we want to have a way to reuse the past computations we had done to compute the key representation of the previous tokens and the value representation of the previous tokens. Does that make sense as a goal?

Does everyone agree? OK. And more precisely, basically, we want to find a way for the key and value matrices to be saved. And I'm not underlining the Q part here because of course, the query corresponding to the previous tokens are not going to be of interest for the present token. So it's like the key and value part. OK. So there is this concept of KV cache, where the goal is to store this key and value matrices somewhere, and reuse them directly from the cache. So what I just mentioned here, when you want to compute a given token, you would reuse the key and the value directly from the cache instead of computing it again. So this is basically, representation of what we just mentioned here. Does that make sense? OK. Awesome. So what else could we do? Yeah. Yeah. Great question. So during training, do we do KV caching? I would say during training, you have this concept of teacher forcing, where you put all your inputs and pass it all together at once. So this concept of KV caching doesn't even come up. Yeah. But great one. Yep. Yep. Yep. Yep. Yeah so the question was, what is the point of all this? We want to just reuse the computations that were already done. And then your answer is yes. Spot on. Any other questions? OK, great. So now that we have seen that we could cache these quantities. So what else could we do? So is there something from last week's lecture that we could reuse, maybe something that reduces the amount of cache, the key and value? Do we have some techniques? Sparse attention. Yeah, that could be. That could be, but not here. Let's say you're attending to everyone and you focus on the number of keys and values. Is there a way to reduce this number? So just to recap, we have here, I'm operating in the context of multi-head attention, which is you have your h heads. So you have h number of query projections, h key projections, h value projections. These are any technique from last week that we could reuse that could reduce this number. Yeah. I heard it. Yeah. Group query attention. Yeah, exactly. So we had these concept of grouping keys and values together, where basically the general formulation is called group query attention, where you group these, and then the external values h and 1 denotes respectively the full multi-head attention, and then multi query attention. And then in modern day LLMs, a lot of papers, they use GQA, with some group that is sensible value. So this is also something we can do. OK, great. And now I'm going to go a little bit into the hardware side of things. So if we were to store the cache for all these values in a naive way, at inference time. So let's say you receive-- so let's say I'm an inference server, I receive queries from multiple users. And let's say I receive a query one and I receive a query two,

PagedAttention, MLA

a naive way of storing the key and value matrices would be in reserving a portion of the memory at the beginning that correspond to the entire context length, because you

don't where you're going to stop. You're decoding because you stop your decoding when you hit the EOS token. So you would have this whole block of size context length, maximum context length that could be reserved. So one observation is that it leads to a lot of waste. So let's say if your context length is of size like 2k, you have 2k reserved for a given request 2k there. So at any given time, let's say, you want to get more requests, you want to serve more requests. Your memory will very quickly not have enough space to accommodate Does these constraints or this challenge make sense? Yeah. OK. And then there is this paper that is called PagedAttention. And it defines several quantities. So there is this reserved quantity internal fragmentation and external fragmentation. Internal fragmentation is basically the space that is being reserved by the model to complete the request. And the reserved spot is the subset of it that corresponds to tokens that were actually being used. So it's the kind of space that's actually used internal fragmentation. There is nothing in there, but it was reserved. And external fragmentation is the memory management system that doesn't necessarily put one block one after another, but has its own way of allocating memory blocks. So it might leave some other gaps. And they came up with a system that is called a PagedAttention that is powering one inference package called the vLLM that solves this issue by mapping, like the generation process by blocks of fixed size. So instead of allocating a whole portion of the memory to answer a given request, it breaks it down by smaller chunks. So I think the paper takes like a value of 16 for blocks. So as you can see, let's say you are in a generation process and you have used a given row, you would just start using another row without wasting more space than that. And then you have some dictionary that maps token position indices to basically their cache values. So there is a smart way of managing it. And they show that it reduces fragmentation by quite a lot. Yeah. OK, great. Now let's do something else with this KV cache. So we just saw that you have your internal representation of your token and you project it into query, key, and value. And then key and value is actually something that is of a dimension that is like non-negligible. And you need to store each copies of it in each transformer block. So you have many of these vectors that you need to keep track of. And is there a way to make that representation somehow more compact? And this is a topic that DeepSeek has tried to tackle in a concept called-- I think it was called multi-latent attention. We're going to see-- so let's look at the vanilla case. If you are in multi-head attention, then for a given representation of a token, you have h projection matrices that turn it into a key. Does everyone agree with that? So the kind of self-attention formula, you have a projection matrix wk for keys, that will project the token representation into some key. And you have h heads. You have h such projections and you have h such resulting embeddings that you need to store in your cache. Because you have that for keys, you have that for values. That's a lot of vectors to store. And these keys and values, they are quite long

as well. So let's see if we can make it smaller. What multi-latent attention does is that it factorizes these projection matrix into an intermediary space that is of lower dimension than the space of keys. You have a first transformation that reduces the dimension and then another one that decompresses it. So this is the first operation that they did in order to make the representation more compact. But they did something else that is very smart. They said that the compression matrix could be shared across keys and values, which says for a given token representation, the cache that you need to store for keys and for values is the same. And then you would have still different matrices here for the decompression part to actually learn different representations for a key and value. But one simplification here has been to share it for across keys and values. And even better, share it across the h heads. So instead of having h such embeddings, you have just one. And then you do that for key. You do that for value. So at the end of it, for every transformer block, you have a single representation per token. Does that make sense? So not only you have less things to store. The DeepSeek V2 paper also showed a performance improvement that it's kind of credited some regularization consequence of doing so. It learns representations that are maybe more useful and shared across keys and values. So apparently there is also performance benefits to it that is not just hardware based. OK, great. So now we have seen some-- Yeah. Yeah. So the question is, do you tune the low rank dimension? So the dimension itself is a design choice. It's fixed. So the question is, is it the same lower dimension for everyone. So the dimension size is fixed and is the same for everyone. I think. But it's a design choice. You could have different ones, but I think in practice it might be the same one. Yeah. Any other questions? OK, great. So now we have less than 10 minutes. We are going to cover techniques that operate at the output token level. And then start with a concept called speculative decoding. That is a very interesting method that uses some smaller model to help the generation of a bigger model, and then uses some scheme that makes the distribution outputs match one of the target one. So it uses a combination of tricks of using a smaller model to generate things, and then gates it with some mathematical based properties to ensure we get a distribution that looks the one of the big model. So here's what is the lectures, the mindset of it. So let's just come back to our favorite example about teddy bears. So what you do is that you ask a smaller model, that the paper calls draft, to generate next words. So you have my teddy bear. What could be next words? So you have one pass. You find token is. You fit token is here again. You find cute and smart. Since the LLM is small, this is typically done faster than with the big LLM. And then once you have all of these, you put the tokens that are predicted by the smaller LLM all as inputs to the target's LLM. That is like the bigger model, like the one that you use, to generate probability distributions for each of them. And then we're going to see that by having some rule on the output distribution of each of these tokens, you can simulate a probability

distribution of each token of each next token that looks one of the target models. So here's basically how it goes. So it looks at the probability of the token that the draft model has predicted. So if the probability of the draft model is bigger than the one of the draft model in the draft model's output distribution, then you just accept the token. Otherwise, you have some acceptance rejection mechanism that either accepts the token or rejects it. And let's say everything is accepted. Then in a single pass, you are able to advance by way more tokens. And then if it fails, then basically you resume the process of generation from that good token onwards with some adjustments on the distribution. So one question that you might ask from all of this, this looks like a recipe. Why is that matching the target distribution? So if you start from the law of total probability and write each term with these quantities. Then you will find the target distribution at the end. So it's an exercise that I encourage you to do. And the paper that is attached to the slide here demonstrates it in just a few lines. So it's a very simple proof. And I think it's a very, very interesting one. Does that make sense? Yeah. Yeah. Yeah. The question is, is that rejection sampling? Yeah. So some of you might be familiar with it already. Does that make sense? Yeah. And then one thing that I didn't say is that why did we even put the last token here? We put all the draft tokens, but also the last one that was predicted because by doing a single forward pass with all these draft tokens, you get for free the probability distribution of the token that comes after that you can then just sample from Qk plus 1. So just to give some summary of the rationale. Doing a single pass is as expensive as doing one pass at a time, because at inference time, you're memory bounds back. In other words, computations are not the limiting factor. It's basically the memory that will bottleneck your computation. So you want to do it on a smaller model. And then on the big model, you want to compute all of these probability distributions at once. Does that make sense? OK. Great. And then I want to talk about one last technique that is called multi-token prediction. It's the same-- so by the way, this previous technique was called speculative decoding. And then this one is basically doing the same. So it's doing slightly a different thing. But the draft model is embedded in the same model, which is very powerful. So basically, it's a model that attaches multiple heads on top of the representation of the last decoder. And the process here is that at training time, it doesn't do next word prediction. It does multi word prediction. And at test time, the draft model is all your heads. And your main model is just the first head. So you have all your draft tokens that you then feed back to the first head and then do acceptance rejection. So the paper does it in a greedy way. So it doesn't do it with this formula. Because since the architecture and then the objective function has changed, you don't have this nice property of finding again, the next token predictions distribution. But you have a slight variation of it. And yeah. So like the interesting notes out of the multi token prediction is the change in objective function. And then second, the fact that the draft model and the target model

are basically embedded in the same one. So don't have to separate the two. And yeah. So you do remember this slide? We have explored techniques that can remedy each of these aspects. So yeah. I highly recommend taking a look at the associated papers as well. And with that, thank you very much.